

§1 Representação da Trie

§1.1 Tamanho do Alfabeto escolhido

O tamanho do alfabeto (`ALPHABET_SIZE`) foi definido exatamente como 36. Como as especificações do trabalho garantem que os títulos (e o *input* do usuário) serão compostos somente por letras do alfabeto, algarismos e espaços em branco, e como o sistema deve desconsiderar os espaços em branco e se as letras são maiúsculas ou minúsculas, precisamos apenas armazenar informação das 26 letras do alfabeto (a-z) e dos 10 dígitos numéricos (0-9), totalizando 36 caracteres.

§1.2 Mapeamento de Caracteres para o Array de Filhos

Cada nó da estrutura (`TrieNode`) contém um array fixo de ponteiros de tamanho 36. O mapeamento de um caractere normalizado para uma das posições (0 a 35) do array é calculado em tempo constante, através de aritmética baseada nos valores da tabela ASCII:

- **Letras minúsculas (a a z):** São mapeadas para a faixa inicial de índices de 0 a 25, subtraindo-se o caractere base 'a'. A expressão fica `index = c - 'a'`.
- **Algarismos numéricos (0 a 9):** São deslocados para preencher as posições restantes (26 a 35). O mapeamento é obtido substituindo o caractere base 0 e adicionando o deslocamento do primeiro bloco: `index = c - '0' + 26`.

§1.3 Conversão de Títulos e Prefixos para Chaves de Busca

O método `toSearchKey(std::string text)` é responsável por “filtrar” os textos. Percorremos caractere por caractere de `text`, modificando caracteres maiúsculos por minúsculos (`c - 'A' + 'a'`) e ignorando espaços. Por exemplo, a string `"Mine Craft2"` passa a ser `"minecraft2"` após o filtro.

§2 Funcionamento dos Métodos

§2.1 Método insert

O método `insert (Game* game)` é responsável por indexar um jogo. O método extrai o título original, computa sua chave de busca e inicia uma descida a partir da raiz da árvore. Para cada caractere da chave, calcula-se o índice correspondente no alfabeto. Caso o ponteiro associado no array `children` seja nulo, um novo `TrieNode` é alocado dinamicamente. O ponteiro de navegação avança para o nó filho correspondente. Ao processar o último caractere da string, o nó terminal alcançado recebe a marcação `isEndOfTitle = true` e armazena o endereço de memória do objeto original contido no vetor global, evitando replicações de memória.

§2.2 Método contains

O método `contains(std::string title)` Determina a existência exata de um título. Transforma o termo fornecido em sua chave de busca interna e navega verticalmente pelos nós da Trie através dos índices calculados. Se a navegação interceptar qualquer ponteiro nulo (`nullptr`) antes do término da chave, deduz-se que a palavra nunca foi indexada, retornando `false` imediatamente. Caso todo o caminho seja percorrido com sucesso, o método avalia e retorna o estado do atributo booleano `isEndOfTitle` do nó

final, impedindo falsos positivos ao buscar por termos que existem na árvore apenas como prefixos de palavras maiores.

§2.3 Método autocomplete

O método `autocomplete(std::string prefix, int k)` é dividido em 3 partes: Primeiro, transforma o prefixo na chave de busca e percorre a árvore até posicionar um ponteiro auxiliar sobre o nó que representa o caractere final do prefixo informado. Se o caminho falhar no meio, encerra retornando um vetor vazio.

Depois, a partir desse nó de parada, invoca-se o método privado auxiliar `recursive_find`. Este realiza uma DFS em toda a subárvore ancorada sobre o prefixo, empilhando todos os ponteiros de jogos (`Game*`) cujos nós terminais possuam `isEndOfTitle` como `true`.

Por fim, A coleção resultante é enviada para o método `sortResults`, que aplica um algoritmo de ordenação conforme os critérios de popularidade decrescente e desempate alfabético das chaves. Finalmente, o sistema extrai as primeiras k posições do vetor ordenado e as retorna.

§3 Análise de Custo Computacional

- Método `insert` ($O(L)$): Depende unicamente do tamanho do título após o filtro, pois possui um loop sobre a string (`for (char c : name)`), onde toda operação dentro e fora do loop é $O(1)$.
- Método `contains` ($O(L)$): O mesmo raciocínio acima se aplica, pois temos um método que realiza uma iteração linear estrita sobre a string. Temos um loop sobre a string, e operações lógicas em cima de cada caractere, que custam $O(1)$.
- Método `autocomplete` ($O(r^2 + p + s)$): Note que

$$T(\text{autocomplete}) = O(p) + O(36 \cdot s) + O(r^2),$$

onde o termo $O(p)$ representa o percurso linear inicial para rastrear o nó raiz do prefixo; o termo $O(s)$ representa a varredura recursiva (`recursive_find`) de s nós internos pertencentes àquela subárvore, e em cada nó o algoritmo varre o array de filhos de tamanho constante (`ALPHABET_SIZE = 36`); o termo $O(r^2)$ vem da complexidade do Insertion Sort, o algoritmo implementado para ordenar o número total de jogos compatíveis que começam com o prefixo dado (r).

§4 Comparação e Uso de Memória

§4.1 Trie vs. Solução Ingênua

Uma solução ingênua armazenaria a coleção de jogos em um array ou lista convencional de tamanho N . Diante de uma requisição de autocomplete, a abordagem ingênua precisaria realizar um *loop* linear completo de 0 a $N - 1$, aplicando testes de comparação de string em cada elemento. Essa estratégia implicaria em um custo temporal de pior caso de $O(N \cdot p)$.

Em sistemas que operam sobre bases massivas de dados (onde N é muito grande), a varredura linear torna-se inviável para aplicações de tempo real. A Trie reduz esse custo drasticamente para valores proporcionais apenas ao tamanho do termo pesquisado e à quantidade de ramificações válidas ($p + s$), operando de forma completamente independente de N na sua etapa de localização.

§4.2 Custo de Memória da Trie

Enquanto a solução ingênua exige apenas $O(N)$ para armazenar os dados, a Trie apresenta um custo de memória grande devido à natureza de seus nós. Cada nó (`TrieNode`) aloca um array estático de 36 ponteiros (tamanho do alfabeto). Em arquiteturas modernas de computação de 64 bits, cada ponteiro consome 8 bytes, totalizando um gasto fixo de $36 \cdot 8 = 288$ bytes por nó isolado exclusivamente para manter o controle de conectividade da árvore, desconsiderando as variáveis internas (`isEndOfTitle` e o ponteiro de dados `game`).

§4.3 Redução de Espaço por Compartilhamento de Prefixos

O fator que viabiliza o uso prático da Trie e diminui seu custo espacial é a propriedade de compartilhamento de prefixos. Títulos que possuem as mesmas sequências iniciais de caracteres dividem os mesmos caminhos na árvore. Por exemplo, ao armazenar os títulos “hades”, “halo” e “halflife”, a sequência comum “ha” é representada por uma única cadeia de dois nós na memória. Quanto maior o volume de palavras inseridas que compartilham prefixos comuns, maior será a taxa de sobreposição dos caminhos e, por conseguinte, menor será o ritmo de criação de novos nós.

§4.4 Impacto do Tamanho do Alfabeto no Espaço Ocupado por Nós

Seja $\mathcal{A}$ o alfabeto utilizado. Como mencionado na seção 4.2, cada nó (`TrieNode`) aloca um array de $|\mathcal{A}|$ ponteiros. Ou seja, a memória ocupada por cada nó será de $8|\mathcal{A}|$ bytes. Portanto, quanto menor conseguirmos tornar o alfabeto, melhor será a *memory management* do nosso sistema.